

Prof. Stefan Göller
Anna Maiworm

Einführung in die Informatik

WiSe 2025/2026

Hausaufgabenblatt 07, Abgabe am 15.12.25 um 12:00 Uhr

Allgemeine Hinweise

- Wenn die Hausaufgabe exakte Benennungen von Dateien, Funktionen, Variablen, etc. vorgibt, dann müssen Sie sich an diese Vorgabe halten. Falls nicht wird Ihre Abgabe nicht oder nur mit Punktabzügen korrigiert.
- Gleiches gilt, wenn die Aufgabe konkrete Anweisungen für `input()` oder `print()` vorgibt.
- Ihr Programm darf keine zusätzlichen `print()` Befehle enthalten, welche in der Aufgabe nicht gefordert sind. Ist eine Ausgabe mit `print()` gefordert, so geben Sie, sofern nicht anders vorgegeben, nur das geforderte Ergebnis und **keinen** zusätzlichen Text aus.
- Benutzen Sie **keine Python-Imports**, außer dies ist in der Aufgabenstellung explizit erlaubt.
- Beachten Sie die Anweisungen, die ggf. zusätzlich in den jeweiligen Aufgaben gegeben sind.
- Geben Sie für jede Aufgabe eine separate Datei ab. Benennen Sie diese Datei als `bXXaY.py`, wobei XX die Blattnummer und Y die Aufgabennummer ist.
- Schreiben Sie die Namen aller Gruppenmitglieder als Kommentar in die erste Zeile jeder Datei.
- Programmcode muss möglichst einfach und gut lesbar sein, um die Korrektur zu erleichtern. Code, der nicht oder nur schwer lesbar ist, kann zu Punktabzug führen.

Allgemeine Bearbeitungshinweise:

- In diesem und in allen folgenden Aufgabenblättern bezeichnet *positive* Zahl eine Zahl die echt größer als 0 ist. Eine *nicht-negative* Zahl ist hingegen eine Zahl die größer oder gleich 0 ist.
- Generell dürfen Sie in den Hausaufgaben, wann immer sinnvoll, eigene Hilfsfunktionen definieren, um die Aufgaben zu lösen.

Hausaufgabe 1 (4 + 3 = 7 Punkte):

Wie in der Vorlesung besprochen, können Sie in Python eine Liste `L` durch den Befehl `L[:]` klonen. Dies klonet jedoch nur die äußere Liste `L` und nicht die Elemente von `L`, die selbst auch veränderlich sind: Folgendes Python-Programm

```
L=[1,3]
K=[L,[2]]
U=K[:]
print(U)
```

klont zwar die Liste `K` und liefert die Ausgabe `[[1,3],[2]]`, falls Sie diesem Programm jedoch die folgenden beiden Zeilen

```
L.append(4)
print(U)
```

anhängen würden, erhielten Sie die Ausgaben `[[1,3],[2]]` und `[[1,3,4],[2]]`. Die Unterliste `L` hat also einen Seiteneffekt (der Tiefe 2) erzeugt, da die Unterliste `[1,3]` in `K` selbst nicht geklont wurde. Genauso können u.U. Seiteneffekte in Tiefe 3 auftreten, indem Sie eine Liste modifizieren, die in einer Liste vorkommt, die selbst wiederum in einer Liste vorkommt (und so weiter).

Eine *int-Superliste* ist rekursiv folgendermaßen definiert:

- Jede Liste, deren Elemente alle vom Typ *int* sind, ist eine int-Superliste.
- Jede Liste deren Elemente alle int-Superlisten sind, ist eine int-Superliste.

- (a) Schreiben Sie eine rekursive Funktion `int_superliste`, die als Eingabe eine Liste erwartet und überprüft, ob es sich dabei um eine int-Superliste handelt, also `True` zurückgibt, falls es sich um eine int-Superliste handelt und sonst `False`.

*Hinweis: Z.B. die Liste $[1, [1], 3]$ ist **keine** int-Superliste, denn keine der beiden Bedingungen ist erfüllt. Z.B. die Liste $[[1, 2], [[1], [3, 4]], [3]]$ ist eine int-Superliste.*

- (b) Schreiben Sie eine rekursive Funktion `klonen`, die als Eingabe eine int-Superliste erwartet und einen Klon der int-Superliste zurückgibt, also eine Kopie bei der die oben beschriebenen Seiteneffekte ausbleiben, d.h. es gibt keine Seiteneffekte beliebiger Tiefe.

Achtung: Für diese Aufgabe dürfen Sie nur Python-Befehle verwenden, welche Sie in der Vorlesung kennen gelernt haben. Insbesondere ist die Verwendung des Befehls `deepcopy` **nicht** erlaubt!

Hausaufgabe 2 (3 Punkte):

Bitte lesen Sie sich diese Aufgabe genau durch. Die Försterin vom Habichtswald möchte die Höhe des kleinsten Baums in ihrem Wald berechnen. Sie kennt bereits die Höhe des größten Baums und weiß, dass folgende Regeln für den Habichtswald gelten:

- Wenn ein Baum der Höhe x existiert, dann auch einer der Höhe $x - 34$.
- Wenn ein Baum der Höhe x existiert, dann auch einer der Höhe $\frac{x-11}{2}$.

Schreiben Sie ein Programm, welches die Höhe des größten Baums im Habichtswald, gegeben als positiver Float-Wert, als Eingabe erwartet und die Höhe des kleinsten Baums ausgibt. Nutzen Sie dafür eine Funktion, die rekursiv die kleinste Höhe berechnet. Beachten Sie, dass natürlich **keine Bäume der Höhe ≤ 0** existieren.

Hinweis: Beachten Sie, dass Sie immer die Resultate beider Regelanwendungen untersuchen müssen. Für den Eingabewert 34.5 liefert eine Anwendung der ersten Regel einen Baum der Höhe 0.5. Der kleinste Baum im Habichtswald hat für den Startwert 34.5 aber die Höhe 0.375: Da wir wissen, dass es einen Baum der Höhe 34.5 gibt, ergibt sich mit der zweiten Regel, dass es auch einen Baum der Höhe $\frac{34.5-11}{2}$ gibt, also einen Baum der Höhe 11.75. Die erste Regel ist auf diesen Baum natürlich nicht mehr anwendbar. Eine weitere Anwendung der zweiten Regel liefert dann den Baum der Höhe $\frac{11.75-11}{2} = 0.375$, auf den keine Regel mehr anwendbar ist.

Hinweis: Um das Minimum zweier Werte a, b zu erhalten, können Sie `min(a, b)` verwenden.

Hausaufgabe 3 (5 Punkte):

Diese Aufgabe knüpft an Präsenzaufgabe 1 an und spricht weiterhin über Pyramiden der Höhe n , wobei n eine positive Ganzzahl ist.

Eine (n, k) -Sequenz von Pyramiden, wobei $n \geq 1$ und $k \geq 0$ ganze Zahlen sind, ist jene Zeichenkette, die aus mehreren aneinandergehängten Pyramiden entsteht, wobei die Pyramide in der Mitte die Höhe n hat, während links und rechts davon je die $(n//2, k-1)$ -Sequenz von Pyramiden steht. Die Pyramiden sind dabei jeweils durch Leerzeichen getrennt. Beachten Sie dabei, dass eine $(n, 0)$ -Sequenz von Pyramiden genau eine Pyramide der Höhe n ist.

Beispiele:

- Die $(5, 1)$ - und die $(5, 2)$ -Sequenz von Pyramiden sind:

```

      *
     ***
    *****
   *  *  *  *  *
  *** ***** ***

```

und

```

      *
     ***
    *****
   *  *  *  *  *
  *  *** *  ***** *  *** *

```

- Für alle $k \geq 3$ ist die $(8, k)$ -Sequenz von Pyramiden:

```

              *
             ***
            *****
           *  *  *  *  *
          *  *  *  *  *  *  *
         *  *  *  *  *  *  *  *
        *  *  *  *  *  *  *  *  *
       *  *  *  *  *  *  *  *  *  *
      *  *  *  *  *  *  *  *  *  *  *
     *  *  *  *  *  *  *  *  *  *  *  *
    *  *  *  *  *  *  *  *  *  *  *  *  *
   *  *  *  *  *  *  *  *  *  *  *  *  *  *
  *  *  *  *  *  *  *  *  *  *  *  *  *  *  *
 *  *  *  *  *  *  *  *  *  *  *  *  *  *  *  *

```

Schreiben Sie eine rekursive Funktion `pyramide_seq`, die als Eingabe zwei Ganzzahlen $n \geq 1$ und $k \geq 0$ erwartet und die k -Sequenz von Pyramiden der Höhe n , also die (n, k) -Sequenz,

zurückgibt. Betrachten Sie als Basisfälle $n==1$ und $k==0$. Im rekursiven Aufruf werden jeweils beide Parameter verkleinert. Ihre Funktion soll die Funktion `pyramide` aus Präsenzaufgabe 1 benutzen. Diese steht Ihnen im Template in der Datei `b07a3.py` zur Verfügung. Bitte tragen Sie Ihre Lösung in die dort vorgegebene Funktion `pyramide_seq` ein (entfernen Sie zuvor die leere return-Anweisung) und geben Sie diese Datei ab.

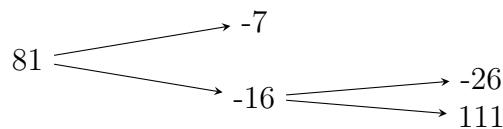
Hausaufgabe 4 (5 Punkte):

Diese Aufgabe knüpft an Präsenzaufgabe 2 an und spricht weiterhin über Integer-Bäume.

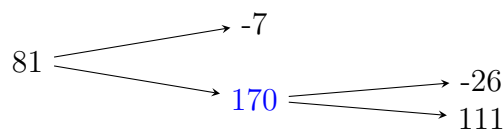
Schreiben Sie eine Funktion `update_tree` die einen Integer-Baum entgegennimmt und mittels Rekursion den Integer-Baum berechnet und zurückgibt, den man erhält, indem man im Eingabebaum jedes Element folgendermaßen aktualisiert:

Jedes Element, das Nachfolger hat (und nur diese!), wird durch das Produkt aus seiner Ebenennummer und der Summe seiner Nachfolger ersetzt. Beachten Sie die Reihenfolge in der die Elemente im Beispiel aktualisiert werden.

Beispiel: Für den Integer-Baum $((111, -16, -26), 81, -7)$, bildlich

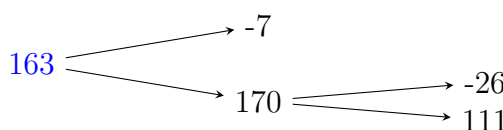


muss die Funktion `update_tree` die Elemente des Baums wie folgt aktualisieren:



Man bekommt einen aktualisierten Integer-Baum $((111, 170, -26), 81, -7)$, indem man die folgende Berechnung durchführt:

$$(111 - 26) \cdot 2 = 170.$$



Man bekommt einen aktualisierten Integer-Baum $((111, 170, -26), 163, -7)$, indem man die folgende Berechnung durchführt:

$$(170 - 7) \cdot 1 = 163$$

Hinweis: Beachten Sie auch hier den Hinweis aus Präsenzaufgabe 2. Auch hier ist es sinnvoll eine solche Hilfsfunktion zu verwenden.

Präsenzaufgaben

Präsenzaufgabe 1:

In dieser Aufgabe untersuchen wir *Pyramiden der Höhe n* , wobei n eine positive Ganzzahl ist. Diese ist eine Zeichenkette, die ein Rechteck der Länge $2 \cdot n - 1$ und der Höhe n darstellt, das in Zeilennummer i mit einer zentrierten Reihenfolge von $2 \cdot i - 1$ Symbolen "*" gefüllt wird, und überall sonst mit Leerzeichen gefüllt ist. Zum Beispiel ist die Zeichenkette "*" die Pyramide der Höhe 1 und " * \n *** \n*****" die Pyramide der Höhe 3, die das folgende Rechteck darstellt:

```
  *
 ***
*****
```

Schreiben Sie eine rekursive Funktion `pyramide` die eine positive Ganzzahl n entgegennimmt und die Pyramide der Höhe n zurückgibt.

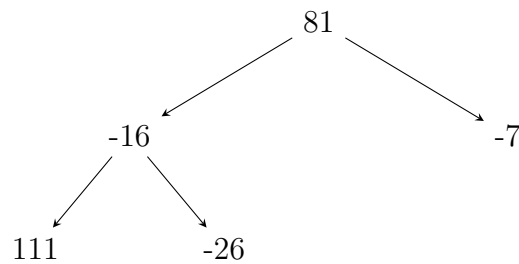
Präsenzaufgabe 2:

Ein Baum ist ein, in der Informatik häufig genutztes, Konzept um strukturierte Daten zu repräsentieren. In dieser Aufgabe behandeln wir sogenannte *Integer-Bäume*, die wie folgt induktiv mit Hilfe von Tupeln definiert sind:

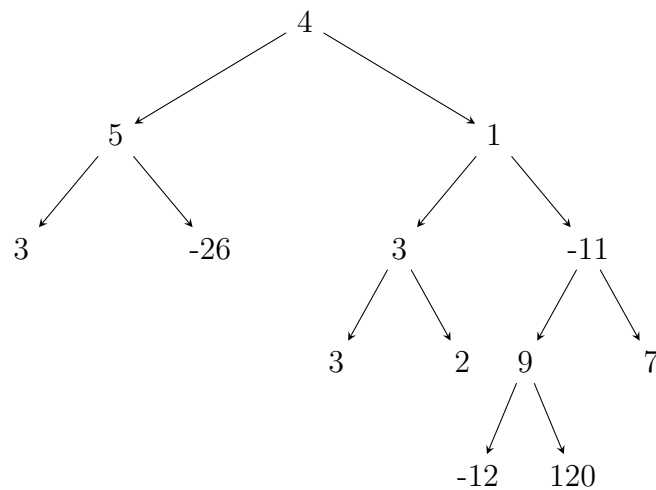
1. Jeder Integer ist ein Integer-Baum.
2. Wenn b_1 und b_2 Integer-Bäume sind und n ein Integer, dann ist auch das Tupel (b_1, n, b_2) ein Integer-Baum.

Beispiele:

- $((111, -16, -26), 81, -7)$ ist ein Integer-Baum. Graphisch kann man sich diesen wie folgt vorstellen:



- $((3, 5, -26), 4, ((3, 3, 2), 1, ((-12, 9, 120), -11, 7)))$ ist ein Integer-Baum. Graphisch kann man sich diesen wie folgt vorstellen:



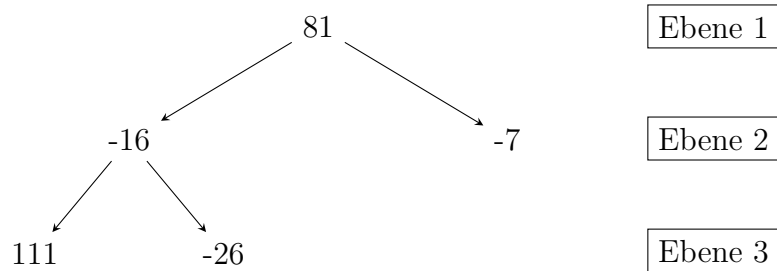
Schreiben Sie eine Funktion `summiere`, die die Zahlen in einem gegebenen Integerbaum, multipliziert mit der jeweiligen Ebenennummer, aufsummiert und zurückgibt. Sie sollen dieses Problem mittels Rekursion lösen.

Beispiele:

- Für den Integer-Baum

$((111, -16, -26), 81, -7)$

bildlich



soll die Funktion also die Summe

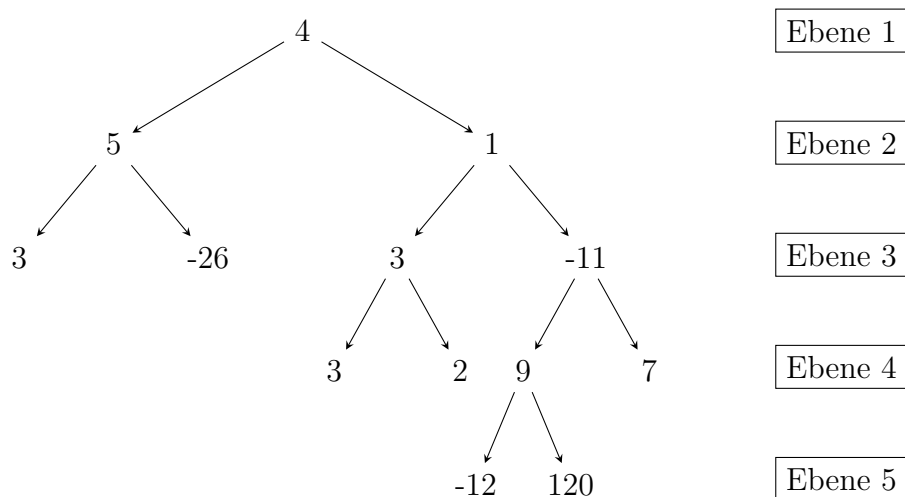
$$1 \cdot 81 + 2 \cdot (-16 - 7) + 3 \cdot (111 - 26) = 290$$

berechnen.

- Für den Integer-Baum

$$((3, 5, -26), 4, ((3, 3, 2), 1, ((-12, 9, 120), -11, 7)))$$

bildlich



soll die Funktion also die Summe

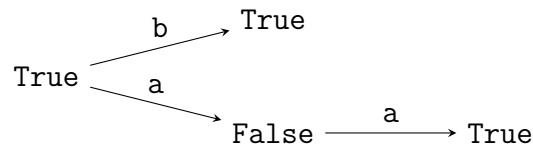
$$1 \cdot 4 + 2 \cdot (5 + 1) + 3 \cdot (3 - 26 + 3 - 11) + 4 \cdot (3 + 2 + 9 + 7) + 5 \cdot (-12 + 120) = 547$$

berechnen.

Hinweis: Es ist sinnvoll, eine weitere Hilfsfunktion zu schreiben, der sowohl der Baum als auch die Ebene übergeben werden. Diese Funktion soll rekursiv vorgehen. Um Ihre Funktion zu testen, legen Sie sich selber in einer Variablen Integer-Bäume, wie oben definiert, an und übergeben Sie diese Ihrer Funktion.

Präsenzaufgabe 3:

Ein Präfixbaum (oft auch “Trie” genannt) ist eine effiziente Datenstruktur zur Speicherung ungeordneter Mengen von Strings. Die Wurzel eines Präfixbaums repräsentiert den leeren String "", und die Kanten zeigen an, welche Zeichen als nächstes folgen können. Jeder Knoten speichert die Information, ob das entsprechende Wort in der Menge ist oder nicht. Zum Beispiel sieht Präfixbaum für ["", "aa", "b", "aa"] wie folgt aus:



Die Bäume werden dargestellt durch “Adjazenzlisten”. Das heißt jeder Knoten ist ein Zweitupel aus einem Boolean, der angibt, ob der Knoten ein Wort der Menge ist, und einer Liste der ausgehenden Kanten. Jede Kante ist ein Zweitupel aus dem Zeichen, das mit der Kante assoziiert ist, und dem Knoten, der dadurch erreicht wird. Für das Beispiel von oben ergibt sich also die folgende Darstellung:

```
(True, [
    ("a", (False, [("a", (True, []))])),
    ("b", (True, []))
])
```

Mit den folgenden Aufgaben soll so eine Baumstruktur aufgebaut und verwendet werden, um aus einer Liste von Strings Duplikate zu entfernen und die Liste zu sortieren. Dafür sollen Methoden wie `sort` und `sorted` **nicht** verwendet werden.

- (a) Schreiben Sie eine rekursive Funktion `contains(tree, word)`, die genau dann `True` ausgibt, wenn der Präfixbaum `tree` das Wort `word` enthält.
- (b) Schreiben Sie eine rekursive Funktion `get_all(tree)`, die alle Wörter des Präfixbaumes `tree`, als Liste ausgibt.
- (c) Nutzen Sie die, in der Datei `add_word.py` gegebene, Funktion `add_word(tree, word)` und Ihre Funktion `get_all(tree, prefix)` um eine Funktion `sort(words)` zu implementieren, die eine Liste von Wörtern als Eingabe bekommt und eine sortierte Liste dieser Wörter zurückgibt. Dabei sollen Duplikate entfernt werden.

Die vorgegebene Funktion `add_word(tree, word)` erwartet als Eingabe einen Präfixbaum und ein Wort, fügt dem Baum dieses Wort hinzu und gibt den so veränderten Baum zurück. Die Adjazenzlisten sind dabei aufsteigend sortiert.

Präsenzaufgabe 4:

Schreiben Sie eine rekursive Funktion, die einen, als String kodierten, nicht-negativen Integerwert entgegen nimmt und bestimmt, ob dieser durch 11 teilbar ist. Benutzen Sie dazu die Äquivalenz folgender Aussagen, die für jede ganze Zahl z gilt:

- z ist durch 11 teilbar
- die alternierende Quersumme von z ist durch 11 teilbar (Also die erste Ziffer minus die zweite Ziffer plus die dritte Ziffer usw.)
- $-z$ ist durch 11 teilbar

Benutzen Sie dafür als Basisfall der Rekursion Strings der Länge 1.

Beispiele: (hier werden die Werte ohne "" dargestellt, im Programm sollen diese aber als Strings vorliegen)

```
14 -> 1 - 4 = -3
    -> 3
    -> False
```

```
7400249944258160101211 -> 7-4+0-0+2-4+9-9+4-4+2-5+8-1+6-0+1-0+1-2+1-1 = 11
                        11 -> 1-1 = 0
                        -> True
```

```
709090939390909090909080705 -> 7-0+9-0+9-0+9-3+9-3+9-0+9-0+9-0
                                +9-0+9-0+9-0+9-0+8-0+7-0+5 = 120
                        120 -> 1-2+0 = -1
                        -> 1
                        -> False
```

Hinweis: Um den Betrag eines Integers z zu erhalten, können Sie $\text{abs}(z)$ verwenden. D.h. z.B. ist $\text{abs}(3) == \text{abs}(-3) == 3$.